

Applications and Releases — A Readable Companion to the OTP Docs

Chapter 7 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why applications and releases exist

A supervision tree answers “what happens when a process fails”. An application answers the same question one level up: what happens when a whole component fails, what it needs before it can start, and where its configuration comes from. A release packages a set of applications with a specific runtime into something you can deploy and boot the same way every time.

Much of this lives in metadata files rather than code, which makes it easy to get subtly wrong. A system that works perfectly in `rebar3 shell` can fail to boot as a release, or boot and then run with half its components dead, because the metadata was never tested the way production uses it.

2. The building blocks

The .app file

Each application is described by an `.app` file (generated from `.app.src` by `rebar3`; from `mix.exs` in Elixir):

Key	Meaning
<code>mod</code>	Callback module and argument: <code>{mod, {my_app, []}}</code> calls <code>my_app:start(normal, [])</code>
<code>applications</code>	Applications that must be running before this one starts. Always at least <code>kernel</code> and <code>stdlib</code>
<code>env</code>	Default configuration values
<code>registered</code>	Names this application registers, used to detect clashes
<code>included_applications, start_phases</code>	Advanced: nesting applications and multi-phase startup

A *library application* has no `mod` key. It provides modules but starts no processes.

Start types

What happens when an application’s top supervisor gives up depends on how the application was started:

Type	If it stops normally	If it crashes
permanent	Whole node stops	Whole node stops
transient	Reported, node keeps running	Whole node stops
temporary	Reported, node keeps running	Reported, node keeps running

`application:start/1` and `application:ensure_all_started/1` default to `temporary`. Applications started from a release's boot script are started as `permanent` by default.

Configuration precedence

From highest to lowest priority:

1. Command line: `erl -my_app key value`
2. `sys.config` in the release
3. `env` in the `.app` file

Read with `application:get_env/2,3`. The docs advise against changing configuration with `set_env` in running production systems.

Releases

A release is described by a `.rel` file: a release name and version, an ERTS version, and the exact version of every application. From it, a boot script (`.boot`) is generated that loads and starts every application in dependency order. In practice `rebar3` (via `relx`) or `mix release` builds all of this for you.

3. How the runtime actually does it

The application controller and masters

Starting an application goes through the application controller. It checks that everything in `applications` is already running, then creates an *application master*, which calls your `start/2`. The master becomes the group leader of every process in the application; that's how the system knows which processes belong to it and where their I/O goes.

Your `start/2` must return `{ok, Pid}` where `Pid` is the top supervisor. When that supervisor exits, the application has stopped, and the start type (above) decides what happens next.

Two versions of every module

The runtime can hold two versions of a module at once: *current* and *old*. Loading new code makes the previous version old; both keep running. Loading a third version purges the old one, and any process still running old code is killed.

Which version a process uses depends on how it calls:

- A fully qualified call (`Module:function()` or `?MODULE:loop()`) always goes to current code.

- A local call (`loop()`) stays in whatever version the process is already running.

OTP behaviours handle the switch for you during a release upgrade, calling `code_change/3` to convert state. Hand-written loops do not.

4. Plausible but wrong

4.1 A dependency missing from applications

```
{application, my_app,
 [{mod, {my_app, []}},
 {applications, [kernel, stdlib]}}].    %% but my_app uses jsx and hackney
```

`rebar3 shell` starts every dependency anyway, so everything works in development. In a release, `hackney` may start after `my_app` or not be included at all, and the first HTTP call crashes with `noproc` or `undef`. Fix: list every runtime dependency in `applications`, and test by booting the actual release rather than the shell.

4.2 Configuration read at compile time (Elixir)

```
defmodule MyApp.Client do
  @timeout Application.get_env(:my_app, :timeout, 5000)
  def fetch(url), do: HTTP.get(url, timeout: @timeout)
end
```

A module attribute is evaluated when the module is compiled, so the value is baked into the `.beam`. Changing it in runtime configuration does nothing. Fix: call `Application.get_env/3` inside the function, or use `Application.compile_env/3` when you really do want compile-time values; Elixir then warns if runtime config disagrees. The Erlang equivalent is putting a config value into a `-define` computed by the build.

4.3 `set_env` as a control channel

```
enable_feature(F) ->
  application:set_env(my_app, F, true).
```

The value is lost on restart, the release's `sys.config` quietly wins again, and any process that read the setting in its `init/1` never sees the change. Fix: keep runtime-changeable settings in an owned store (ETS with a clear owner, or a config service) and restart or notify the processes that depend on them.

4.4 Real work inside `start/2`

```
start(_Type, _Args) ->
  {ok, _} = my_db:connect(os:getenv("DB_URL")),
  my_sup:start_link().
```

The whole node's boot waits on the database. If the database is down, `start/2` crashes, the application fails to start, and with a permanent start type the node exits, often into a restart loop driven by `systemd` or `Kubernetes`. Fix: `start/2` should only start the top supervisor; connections belong in supervised workers that retry (chapter 1, section 4.3).

4.5 A hand-written loop and hot code loading

```
loop(State) ->
  receive
    {work, X} -> loop(handle(X, State))
  end.
```

The local call `loop(...)` keeps the process in the version it started with. After one code load it runs old code; after a second load it is killed. Fix: use a behaviour, or make the recursive call `?MODULE:loop(...)` and design the state so old and new versions can both read it.

4.6 A node that keeps running without its application

```
%% custom startup script
main() ->
  {ok, _} = application:ensure_all_started(my_app),
  timer:sleep(infinity).
```

`ensure_all_started/1` uses the `temporary` start type. When `my_app`'s top supervisor exceeds its restart intensity, the application stops and the node carries on, still answering health checks while doing nothing useful. Fix: boot as a proper release (permanent by default), or call `application:ensure_all_started(my_app, permanent)`, so a dead application takes the node down and your orchestrator restarts it.

5. Review checklist and sources

- ☐ Every runtime dependency is listed in `applications`
- ☐ The system is tested by booting the release, not only in the shell
- ☐ Configuration is read at runtime where it is meant to be changeable
- ☐ `set_env` is not used as a runtime control mechanism
- ☐ `start/2` only starts the top supervisor
- ☐ Start types are chosen deliberately; production applications are `permanent`
- ☐ Long-running processes are behaviours, or make fully qualified recursive calls

Sources

- Applications, OTP Design Principles: <https://www.erlang.org/doc/system/applications.html>
- Releases, OTP Design Principles: https://www.erlang.org/doc/system/release_structure.html
- Compilation and Code Loading, Reference Manual: https://www.erlang.org/doc/system/code_loading.html